

FC 0x65 — Trigger Data Streaming 프로토콜

기능 요약

디바이스 내부 버퍼에 데이터를 쌓아두었다가, 트리거 조건이 성립된 시점을 기준으로 일괄 반환하는 하드웨어 트리거 방식의 데이터 캡처 프로토콜.

FC 0x64 Continuous 방식(통신이 들어오면 그 시점 데이터를 전달)과 달리, 버퍼에 데이터를 축적한 뒤 트리거 발생 시점의 전후 데이터를 한꺼번에 제공한다.

1. 프레임 구조

모든 프레임은 Modbus RTU 형식을 따른다.

[Node ID (1B)] [Function Code = 0x65 (1B)] [Control (1B)] [Payload...] [CRC_L (1B)] [CRC_H (1B)]

Control 코드

Control	방향	기능
0x00	TX → RX	Stop (캡처 중단, 버퍼 초기화)
0x01	TX → RX	Start / Check Status (트리거 대기 시작 또는 상태 확인)
0x02	TX → RX	Configure (파라미터 설정)
0x03	TX → RX	Request Data (버퍼 데이터 요청)

2. Control 0x00 — Stop

트리거 대기를 중단하고 내부 버퍼를 초기화한다.

TX 프레임

Byte	필드	값
0	Node ID	Slave 주소
1	Function Code	0x65
2	Control	0x00
3-4	CRC	—

총 5 bytes

RX 프레임 (Echo)

TX와 동일한 5 bytes 에코 응답.
(*FrameLength* = 2 : FC + Control 2바이트 기준)

3. Control 0x01 — Start / Check Status

처음 전송 시 트리거 대기(MONI 플래그 활성화)를 시작하고, 이후 반복 전송으로 트리거 발생 여부를 폴링한다.

동작 규칙: Flag1.Triggered == 1 상태(이미 트리거 발생)에서는 MONI 플래그를 재설정하지 않는다. 즉, 트리거 발생 후에도 폴링을 계속 보내도 안전하다.

TX 프레임

Byte	필드	값
0	Node ID	Slave 주소
1	Function Code	0x65
2	Control	0x01
3-4	CRC	—

총 5 bytes

RX 프레임

Byte	필드	설명
0	Node ID	—
1	Function Code	0x65
2	Control	0x01
3	Status	0 = 트리거 미발생, 1 = 트리거 발생
4–21	(미사용)	구조체 잔여 필드 18 bytes (무시)
22–23	CRC	—

총 24 bytes

($FrameLength = 21 = FC(1) + Control(1) + Status(1) + \text{미사용}(18). NodeID(1) + 21 + CRC(2) = 24$ bytes)
의미 있는 필드는 Status(byte 3)만. 미사용 영역에는 디바이스 내부 TMON 구조체 값(설정 에코 등)이 그대로 실려 오지만 마스터는 무시한다.

구현 참조: app.js _sendAndReceiveFC65Inner() — control 0x01 은 정확히 24 bytes 수신 후 CRC 검증, modbus.js parseTriggerStatusResponse() — bytes[3] 을 Status 로 사용.
모니터 툴팁에서 CRC 가 Byte 22-23 으로 표시되는 것이 정상이다.

4. Control 0x02 — Configure

트리거 캡처 파라미터를 설정한다. 설정 즉시 내부 버퍼와 상태가 초기화된다 (InitTriggerMonitoring() 호출).

순서: 반드시 0x00 Stop → 0x02 Configure → 0x01 Start 순서로 진행한다.

TX 프레임

Byte	필드	타입	설명
0	Node ID	uint8	Slave 주소
1	Function Code	uint8	0x65

Byte	필드	타입	설명
2	Control	uint8	0x02
3-4	Period	uint16 BE	샘플링 주기. 1 unit = 125 μ s. 예) 1600 $\rightarrow$ 200 ms
5	CH_Sel[0]	uint8	슬롯 0 채널 번호 (미사용: 0xFF)
6	CH_Sel[1]	uint8	슬롯 1 채널 번호 (미사용: 0xFF)
7	CH_Sel[2]	uint8	슬롯 2 채널 번호 (미사용: 0xFF)
8	CH_Sel[3]	uint8	슬롯 3 채널 번호 (미사용: 0xFF)
9	SourceSEL	uint8	트리거 소스 채널 번호 (Chart_Channel_Definitions 기준). 0xFF = Immediate
10	Edge	uint8	0 = Rising $\uparrow$, 1 = Falling $\downarrow$
11	Position	uint8	트리거 시점 위치 0~99 (%). GUI 입력란은 1~99 로 제한
12-15	Level	float32 LE	트리거 레벨 값 (ARM little-endian)
16-17	NumOfData	uint16 BE	총 샘플 수 256~1024 . 범위 벗어나면 클램프
18-19	CRC	—	—

총 20 bytes

CH_Sel 채널 번호

Chart_Channel_Definitions.md 에 정의된 Hex 번호와 동일.

예) 0x00 = Velocity Feedback, 0x03 = Torque Feedback

Position 동작

$TMON.Position(\text{내부}) = (100 - \text{position}) \times \text{NumOfData} / 100$

- position = 0 $\rightarrow$ 트리거 시점이 버퍼 맨 왼쪽 (pre-trigger 없음, 전부 post-trigger)
- position = 25 $\rightarrow$ 버퍼의 25%가 pre-trigger, 75%가 post-trigger

- position = 99 → 버퍼의 99%가 pre-trigger, 1%가 post-trigger

TMON.Position은 post-trigger 샘플 수를 나타낸다.

Immediate Trigger (SourceSEL = 0xFF)

트리거 조건 없이 Configure 직후 즉시 캡처를 시작한다.

Start(0x01) 폴링 시 곧바로 Status = 1 반환.

RX 프레임 (Echo)

TX 프레임을 그대로 에코 반환한다. (20 bytes)

(FrameLength = 17 : FC~NumOfData 17바이트 기준)

5. Control 0x03 — Request Data

트리거 완료 후 채널별 버퍼 데이터를 분할 수신한다.

한 번의 요청으로 최대 **14개 float** 반환. 전체 데이터는 반복 요청으로 수집한다.

TX 프레임

Byte	필드	타입	설명
0	Node ID	uint8	Slave 주소
1	Function Code	uint8	0x65
2	Control	uint8	0x03
3	CH_Sel	uint8	요청할 채널 슬롯 인덱스 (0~3)
4-5	StartAddress	uint16 BE	버퍼 내 읽기 시작 오프셋
6-7	CRC	—	—

총 8 bytes

RX 프레임

Byte	필드	타입	설명
0	Node ID	uint8	—
1	Function Code	uint8	0x65
2	Control	uint8	0x03
3	Status	uint8	0 = 미트리거, 1 = 트리거 발생 (데이터 유효)
4	CH_Sel	uint8	요청한 채널 슬롯 인덱스 에코
5–6	StartAddress	uint16 BE	요청한 StartAddress 에코
7	Len	uint8	이번 응답의 float 데이터 개수 (최대 14)
8–63	Data[14]	float32[] LE	샘플 데이터 영역 (14 × 4 = 56 bytes 고정). 앞에서부터 Len 개만 유효, 나머지는 무의미
64–65	(패딩)	—	항상 미사용 (FrameLength 65 를 맞추기 위한 구조체 잔여)
66–67	CRC	—	—

총 68 bytes 고정

($FrameLength = 65 = FC \sim Len(7) + Data(56) + 패딩(2)$. $NodeID(1) + 65 + CRC(2) = 68 \text{ bytes}$)

Len 값과 무관하게 응답 길이는 항상 68 bytes 이며, 유효 데이터 개수는 Len 필드로만 판단한다.

구현 참조: app.js _sendAndReceiveFC65Inner() — control 0x03 은 정확히 68 bytes 수신 후 CRC 검증, modbus.js parseTriggerDataResponse() — bytes[7] 을 Len, bytes[8..] 을 float32 LE 로 파싱 (최대 14개).

모니터 화면 확인 시 주의: Monitor 탭 메인 라인의 바이트 영역은 overflow: hidden 이라 68 bytes 프레임이 화면 폭에 따라 중간(약 40~50 byte 부근)에서 잘려 보인다. 실제 수신 길이는 해당 엔트리를 클릭해 확장 뷰의 **Frame Length / Raw** 로 확인한다 (CRC 검증을 통과한 프레임만 RX 로 표시되므로, 표시된 프레임은 반드시 68 bytes 이다).

버퍼 순회 방법

```
startAddress = 0
while startAddress < numOfData:
    요청: CH_Sel, StartAddress = startAddress
    응답: Len개의 float 수신
    startAddress += Len
    if Len < 14:
        break ← 마지막 패킷 (버퍼 끝 도달)
```

종료 조건은 두 가지를 함께 사용한다.

- $Len < 14$: 버퍼 끝 도달 (마지막 패킷)
- $startAddress \geq numOfData$: $numOfData$ 가 14 의 배수인 경우 마지막 패킷도 $Len = 14$ 로 옴으로, 누적 개수로도 종료를 판단해야 한다
- $Len = 0$ 응답(범위 밖 StartAddress) 도 종료로 처리한다

GUI 가 허용하는 Samples 값(256 / 512 / 768 / 1024) 은 모두 14 의 배수가 아니므로 마지막 패킷의 Len 은 각각 4 / 8 / 12 / 2 가 된다.

채널은 슬롯 순서대로 ($0 \rightarrow 1 \rightarrow 2 \rightarrow 3$) 개별 수집한다. (비활성 슬롯은 건너뛴다)

6. 전체 동작 흐름



7. 시간 축 계산

수집된 데이터의 타임스탬프는 다음과 같이 계산한다.

$\text{periodMs} = \text{Period} \times 0.125$ (ms 단위, 1 unit = 125 μs)

$\text{preTriggerSamples} = \text{numOfData} \times \text{position} / 100$

$\text{sample}[i].\text{time} = (i - \text{preTriggerSamples}) \times \text{periodMs}$

- $\text{time} < 0$: pre-trigger 구간 (트리거 이전)
- $\text{time} = 0$: 트리거 발생 시점
- $\text{time} > 0$: post-trigger 구간 (트리거 이후)

8. FrameLength 참고사항

FrameLength 는 USB-HID 환경에서 유래한 값으로 RS-485 통신의 실제 바이트 수와 차이가 있다.

Control	FrameLength	485 실제 RX 총 바이트	산출 기준
0x00 Stop	2	5	NodeID(1) + FC+Ctrl(2) + CRC(2)
0x02 Configure	17	20	NodeID(1) + FC~NumOfData(17) + CRC(2)
0x01 Status	21	24	NodeID(1) + FrameLength(21) + CRC(2)
0x03 Data	65	68	NodeID(1) + FrameLength(65) + CRC(2)

FrameLength = FC 바이트부터 시작하는 페이로드 길이 (NodeID, CRC 제외).

0x03 Data의 FrameLength=65는 USB-HID 레거시 고정값. 실제 유효 데이터 길이는 Len 필드로 판단한다.

각 프레임의 CRC 위치(0-based) = 총 바이트 - 2 : Stop 3-4, Configure 18-19, Status 22-23, Data 66-67.